Open Source for development purposes.

Download Corda: Visit *https://github.com/corda/corda* or *https://r3.com/get-corda/* and download the latest version of Corda.

Corda example projects: You can clone a Corda example project to understand the structure. For example:

```
bash
CopyEdit
git clone https://github.com/corda/samples
cd samples/cordapp-example
```

Step 5: Set up Corda nodes

In Corda, nodes represent the individual participants in the distributed network. Each node runs a Corda service, which interacts with other nodes.

Run the Corda network: To run a basic Corda network, you can use Docker. The Corda team provides Docker containers to easily start nodes. For example, you can use the Corda network from the Corda Network Docker GitHub repository.

Install Docker and run the network:

```
git clone https://github.com/corda/corda-docker
cd corda-docker
docker-compose up
```

Running Corda nodes: Once the Docker containers are running, the nodes in the network will automatically connect to each other. You can interact with them using the Corda shell.

Step 6: Create your first Corda CorDapp

A CorDapp (Corda Distributed Application) is the application that you will develop to run on the Corda network.

Create a new CorDapp project: Use the Corda templates or create a new project manually in IntelliJ. To create a new project:

- Open IntelliJ IDEA and create a new project.
- Select Kotlin as the language and configure the project settings.

Define the CorDapp:

- Create a contract that defines the rules of your distributed application (business logic).
- Create a flow that defines how the participants interact with each other.
- Write tests to simulate the interaction between nodes.

Here's an example structure for a CorDapp:

```
my-cordapp
├── build.gradle
├── src
│   ├── main
│   │   ├── kotlin
│   │   │   ├── com
│   │   │   │   ├── example
│   │   │   │   │   ├── contract
│   │   │   │   │   └── flow
│   │   │   │   ├── util
│   │   └── test
│   │       └── kotlin
```

Example flow: Here's a basic example of how a flow may look in Corda:

```
@StartableByRPC
@InitiatingFlow
class ExampleFlow : FlowLogic<String>()
{
    @Suspendable
    override fun call(): String {
        return "Hello, Corda!"
    }
}
```

Create a contract: Contracts define the rules that govern transactions in Corda. Here's a simple contract example:

```
class ExampleContract : Contract {
    companion object {
        const val ID = "com.example.
contract.ExampleContract"
    }
    override fun verify(tx:
TransactionForVerification) {
        // Custom verification logic
    }
}
```

Step 7: Deploy the CorDapp

Once your CorDapp is ready:

Build the CorDapp: Run the following Gradle command to build the project:

```
./gradlew build
```

Deploy the CorDapp: The CorDapp *.jar* files need to be deployed to the nodes. You can copy them into the corda-apps directory of each node.

Run the flow: Use the Corda shell to invoke your flow:

```
run flowExample.ExampleFlow
```

Step 8: Interact with the network

You can interact with the network using the Corda shell or through a web interface. The Corda shell allows you to issue commands like sending transactions, invoking flows, and viewing node information.

Successfully configuring the Corda platform paves the way for creating and releasing distributed applications (CorDapps) that are safe, scalable, and privacy-preserving. We have established a solid foundation for permissioned blockchain solutions by setting up the notary and network map service, configuring the network parameters, and installing Corda nodes. This configuration allows for smooth peer-to-peer communications that are both secure and compliant with enterprise-level standards. END

By: Dr R. Anand

The author is working as a professor in the Department of Computer Science and Engineering at Easwari Engineering College, Chennai, Tamil Nadu.